

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedys vs Dinámico vs Bruto.»

Isidora Sofía Villegas Hernández

24 de junio de 2026

19:59

1. Implementaciones

En esta sección se describen los algoritmos implementados para resolver el problema de optimización de satisfacción en el visionado de animes bajo restricciones de tiempo y energía.

1.1. Algoritmo de Fuerza Bruta

Se implementó una búsqueda exhaustiva mediante **Backtracking**. El algoritmo explora recursivamente todas las combinaciones posibles de prefijos para cada anime.

- **Lógica:** Para cada anime i , se prueba con no ver ningún capítulo o ver un prefijo de longitud $k \in \{1, \dots, q_i\}$. Si los recursos (tiempo M y energía E) no se agotan, se procede al siguiente anime.
- **Complejidad Temporal:** $O(\prod_{i=1}^n (q_i + 1))$. Debido a su crecimiento exponencial, este algoritmo solo es viable para instancias pequeñas y medianas ($n \leq 80$).
- **Complejidad Espacial:** $O(n)$, correspondiente a la profundidad máxima del árbol de recursión.

1.2. Programación Dinámica

Se modeló el problema como una variante del **Problema de la Mochila Multidimensional y de Múltiple Opción** (MMKP).

- **Estado:** $DP[m][e]$ representa la máxima satisfacción obtenible con exactamente m minutos y e unidades de energía.
- **Transición:** Para cada anime, se actualiza el estado considerando cada prefijo posible de manera excluyente (solo se puede elegir un prefijo por anime). Para evitar usar el mismo anime varias veces en una misma iteración, se utiliza una copia temporal del estado previo.

- **Complejidad Temporal:** $\mathcal{O}(n \cdot M \cdot E \cdot q_{\max})$. Con $n = 200$, $M = 3000$, $E = 500$, $q_i = 30$, el total de operaciones es manejable en tiempos modernos ($\approx 9 \times 10^9$ operaciones teóricas, pero mucho menos en la práctica por las restricciones de recursos).
- **Complejidad Espacial:** $O(M \cdot E)$ mediante la técnica de optimización de espacio.

1.3. Heurísticas Greedy

Se implementaron dos enfoques voraces con criterios locales distintos:

1.3.1. Greedy 1: Siguiente Capítulo

Este algoritmo avanza 'capítulo a capítulo'. En cada paso, evalúa el siguiente capítulo disponible de todos los animes.

- **Criterio:** Maximizar el ratio $v_{i,j}/(t_{i,j} + c_{i,j})$. Si el capítulo completa el anime, se suma el bono b_i a la satisfacción antes de calcular el ratio.
- **Complejidad:** $O(Q \cdot n)$, donde Q es la cantidad total de capítulos.

1.3.2. Greedy 2: Mejor Prefijo Global

A diferencia del anterior, este evalúa todos los prefijos posibles de los animes restantes en cada paso.

- **Criterio:** Seleccionar el prefijo de capítulos ($1 \dots k$) que maximice el ratio de satisfacción acumulada sobre el costo total acumulado: $(\sum v + b)/(\sum t + \sum c)$.
- **Complejidad:** $O(n^2 \cdot \max(q_i))$.

2. Entorno experimental

El experimento consiste en evaluar el rendimiento de los cuatro algoritmos implementados bajo diferentes escenarios. Se generaron 27 casos de prueba divididos en tres categorías:

- **Casos Pequeños** ($n \in \{3, 5, 8\}$): Diseñados para validar que la Fuerza Bruta y la Programación Dinámica coinciden en el resultado óptimo.
- **Casos Medianos** ($n \in \{20, 40, 80\}$): Orientados a medir la brecha de calidad entre las heurísticas Greedy y la solución óptima de DP. De igual manera, para demostrar que fuerza bruta es poco viable, se ejecutó para instancias de $n = 20$ y $n = 40$ únicamente.
- **Casos Grandes** ($n \in \{100, 150, 200\}$): Utilizados para analizar el escalamiento temporal y de memoria de los algoritmos eficientes.

Las pruebas se ejecutaron en una máquina con sistema operativo Linux, midiendo el tiempo de ejecución en milisegundos mediante la librería `chrono` y el uso de memoria máximo (`ru_maxrss`) mediante `getrusage`.

3. Resultados y Análisis

A continuación se presentan los resultados obtenidos tras ejecutar la suite de pruebas.

3.1. Tiempo de Ejecución

La Figura 1 muestra el tiempo de ejecución en escala logarítmica respecto al número de animes n . Este gráfico global muestra la escalabilidad de los algoritmos en una amplia gama de tamaños de entrada.

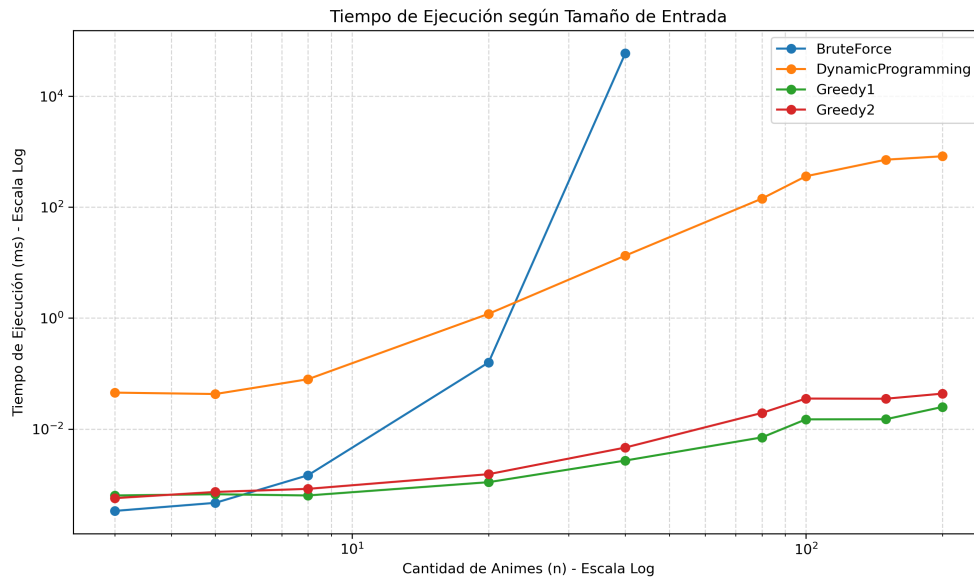


Figura 1: Tiempo de ejecución promedio por algoritmo.

Dado que el algoritmo de Fuerza Bruta posee un crecimiento exponencial, este fue ejecutado en instancias de hasta $n = 40$. Por esta razón, además del gráfico global de tiempo, se incluye un gráfico complementario tipo zoom (Figura 2) que considera únicamente los valores de $n \leq 40$ donde Fuerza Bruta pudo ejecutarse. Esto permite comparar con mayor claridad el comportamiento detallado de los cuatro algoritmos en el mismo rango de entrada y refuerza la validación experimental al comparar directamente Fuerza Bruta y Programación Dinámica.

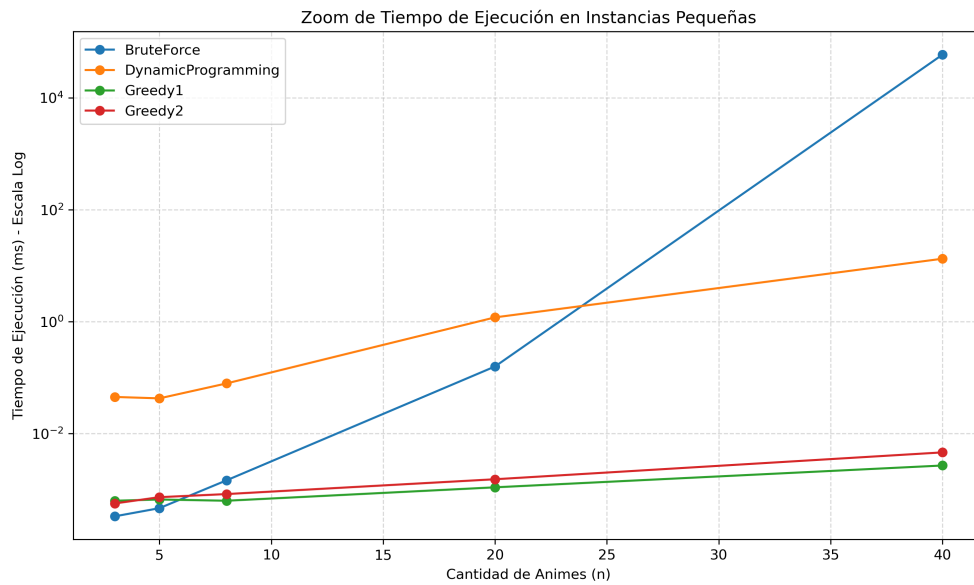


Figura 2: Zoom del tiempo de ejecución en instancias de hasta $n = 40$ donde Fuerza Bruta es ejecutable.

Se observa en la figura global que la Programación Dinámica crece de manera lineal respecto a n cuando M , E y la cantidad máxima de capítulos por anime ($q_{\text{máx}}$) se mantienen fijos. Es importante notar que su complejidad teórica es $\mathcal{O}(n \cdot M \cdot E \cdot q_{\text{máx}})$, por lo que el crecimiento no es estrictamente lineal en todos los parámetros de entrada. Como se mencionó, Fuerza Bruta no es viable para instancias más grandes ($n > 40$) y resulta útil principalmente como línea base de validación. Las heurísticas Greedy son órdenes de magnitud más rápidas, manteniendo tiempos casi instantáneos incluso para $n = 200$.

3.2. Uso de Memoria

La Figura 3 presenta el consumo de memoria máximo durante la ejecución.

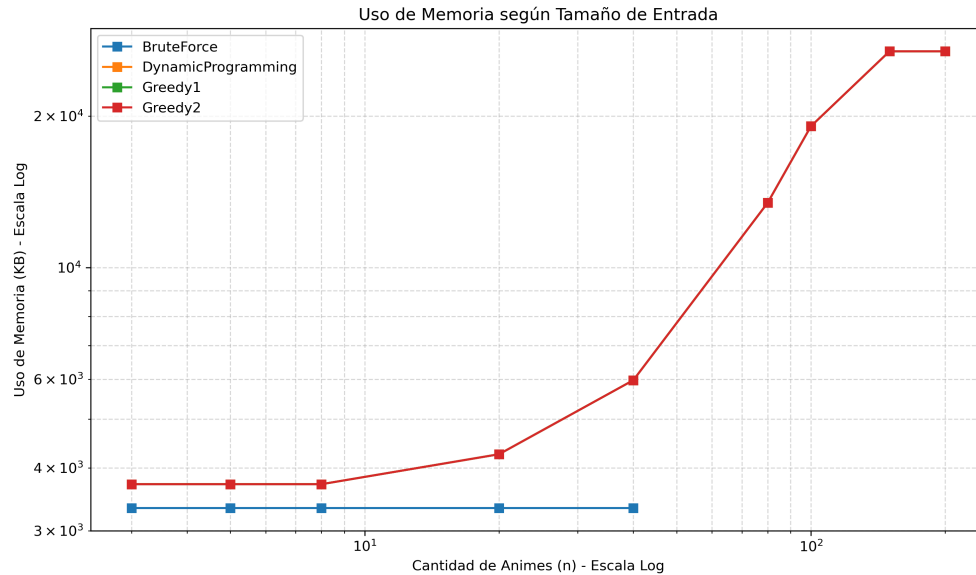


Figura 3: Uso de memoria máximo por algoritmo.

La medición de memoria fue realizada mediante `ru_maxrss`, por lo que representa el máximo uso de memoria alcanzado por el proceso durante la ejecución. Debido a esto, algunas curvas pueden superponerse o reflejar memoria reutilizada entre algoritmos, especialmente si estos son ejecutados dentro del mismo programa principal. Por lo tanto, el gráfico debe interpretarse como una aproximación del consumo máximo observado y no como una medición completamente aislada por algoritmo. Aun así, el consumo de memoria se encuentra fuertemente influenciado por la tabla utilizada en Programación Dinámica, cuyo tamaño depende de $M \cdot E$.

3.3. Calidad de la Solución

La calidad se define mediante el siguiente ratio:

$$\text{Calidad} = \frac{\text{Satisfacción Greedy}}{\text{Satisfacción DP}}$$

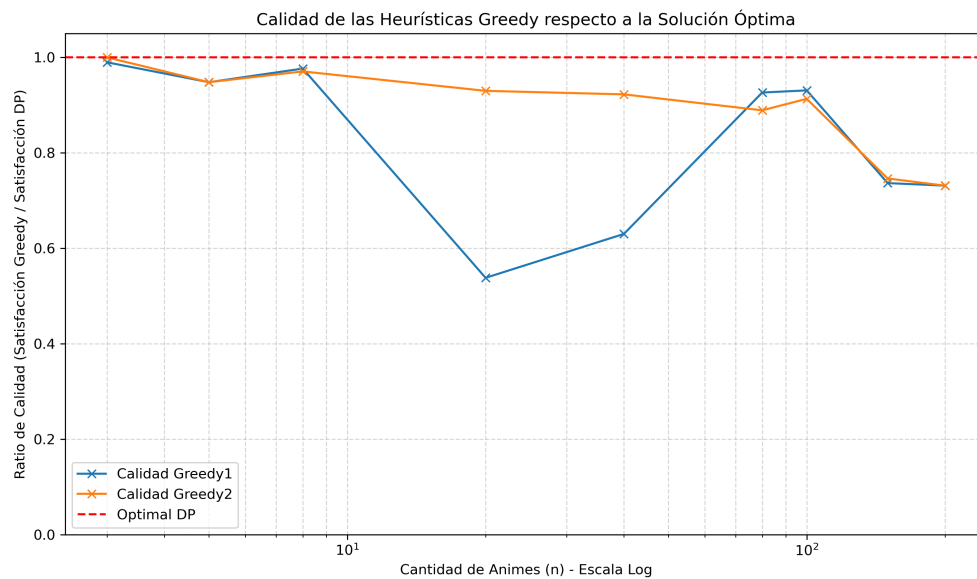


Figura 4: Ratio de calidad de las heurísticas Greedy.

En el gráfico, la línea horizontal en 1.0 representa la solución óptima de DP, y los valores más cercanos a 1.0 indican una mejor calidad de solución. Cabe destacar que el eje Y de este gráfico se mantiene en escala lineal desde 0 hasta 1.05 para facilitar la interpretación del ratio. Se observa que **Greedy 2** (Mejor Prefijo Global) tiende a ofrecer resultados más estables y superiores, ya que al evaluar prefijos completos logra capturar mejor el beneficio de completar un anime (bonos). Por su parte, **Greedy 1** (Siguiendo Capítulo) toma decisiones más locales capítulo a capítulo, lo que lo lleva a fallar o entregar menor calidad en ciertos casos.

4. Conclusiones

Tras el análisis experimental de los cuatro algoritmos propuestos para el problema AniMaraton, se concluye lo siguiente:

1. **Optimalidad vs Eficiencia:** El algoritmo de Fuerza Bruta es útil únicamente para validar la correctitud en casos de hasta $n = 40$, siendo impráctico para instancias de mayor tamaño. La Programación Dinámica logra la solución óptima y funciona de manera altamente efectiva dadas las restricciones actuales, lo que la convierte en la opción preferida si la exactitud es crítica.
2. **Desempeño de Heurísticas:** Las heurísticas Greedy, aunque no garantizan el óptimo, demostraron ser órdenes de magnitud más rápidas. **Greedy 2** se destaca por capturar mejor la estructura del problema al evaluar prefijos completos, logrando una calidad de solución superior y más estable que Greedy 1 en la mayoría de los casos.
3. **Impacto de los Bonos:** El bono de completación es un factor determinante. Los algoritmos que logran "mirar hacia adelante" evaluar el impacto completo del anime (DP y Greedy 2) aprovechan

mejor estos bonos en comparación con decisiones locales capítulo a capítulo (Greedy 1).

4. **Uso de Recursos:** Respecto al uso de memoria, si bien DP requiere un espacio fijo significativo debido a la tabla, las mediciones empíricas mostradas son aproximaciones (influidas por `ru_maxrss` y la reutilización de memoria en el programa principal). Aún así, en sistemas muy restringidos, una heurística Greedy podría ser la única alternativa estrictamente viable.

En conclusión, para el problema AniMarathon, la Programación Dinámica es altamente efectiva y entrega resultados óptimos bajo las restricciones dadas, mientras que Greedy 2 ofrece un excelente balance para casos donde el tiempo de respuesta y los recursos sean los factores más críticos.